GPU-Accelerated Data Collection for Reinforcement Learning

Report: CPU Baseline vs GPU Vectorization vs GPU Kernel Fusion on CartPole Simulation

Based on Gleeson et al., "Optimizing Data Collection in Deep Reinforcement Learning" (MLBench 2022)

Team Members :

Ashwani Bhardwaj (G25AIT2022) | **Yash Mehta (G25AIT2133)** | **Ravikant Pandey (G25AIT2085)** | **Vaibhav Diwedi (G25AIT2122)**

1. Introduction

1.1 Background:

Reinforcement learning helps learn an agent by exploration and exploitation. The agent takes some action and gets rewards and learns the policy. Training alternates between two phases. In the **data-collection** phase, the current policy plays N copies of the simulator for K steps. In the **model-update** phase, the N × K samples gathered this way are used to train the policy network with backpropagation (Figure 1).

They study two solutions:

- **GPU vectorization** stores all N simulator states as one N × S matrix on the GPU, so that every simulator advances in parallel.
- **Simulator kernel fusion** runs K simulation steps, inside a GPU kernel launch. Simulator kernel fusion avoids paying the launch cost K times. Keeps the state fast on-chip storage.

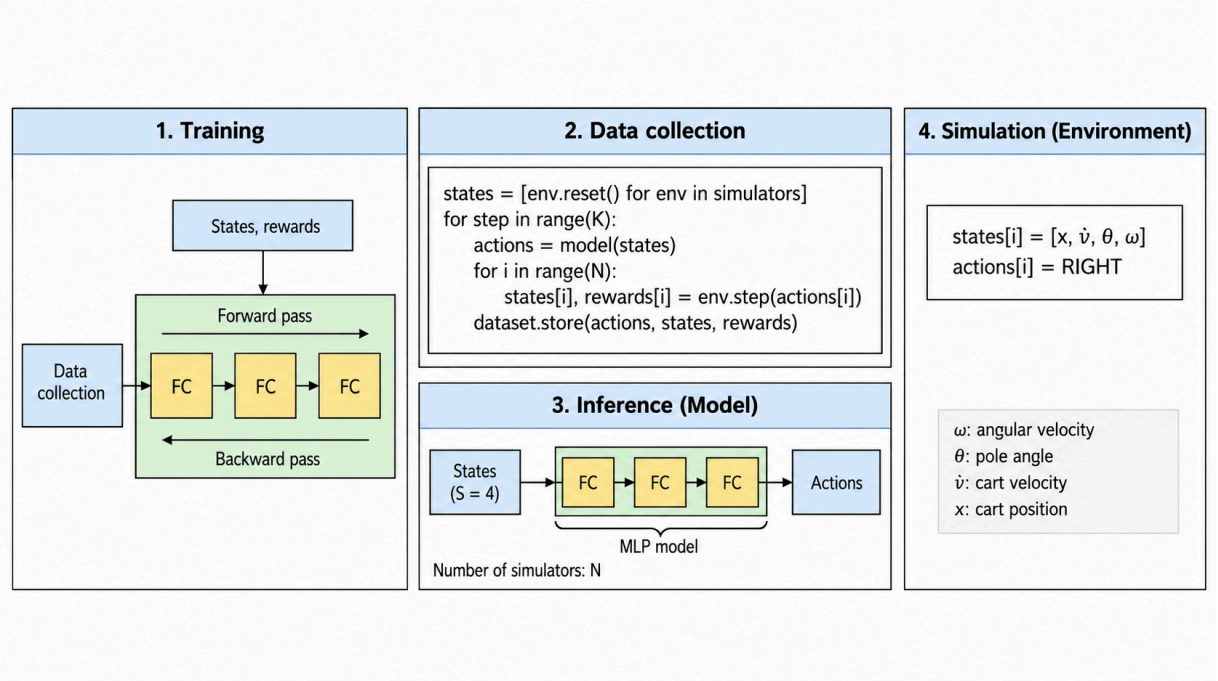


Figure 1: The RL training cycle (left) and the step where training data is gathered from the CartPole simulator (right). N is the number of simulators and K the number of steps per iteration. Reproduced from Gleeson et al. [1], Figure 1.

1.2 Ipython Notebook Overview

This notebook rebuilds the paper's two GPU speed-ups on the CartPole simulator, using only Python and Numba's CUDA support, with no deep-learning libraries. It times a CPU loop, a GPU version with one kernel

launch per step, and a fused GPU version that runs many steps in one launch. It then checks the results match and produces tables, plots, an animation and a live demo.

1.3 Objectives of this report

The report explains the notebook's organization and design, gives a pseudo-algorithm and our observations for each approach.

2. Implementation Overview

2.1 How the notebook is organized

- One notebook with 54 cells:
- Thirteen `%%writefile` cells create files in `src/`, `scripts/`, or `tests/`; outputs go to `results/`.
- Steps 1–2: Install libraries, check GPU, create folders.
- Steps 3–7: Build the simulator and CPU, GPU-vectorized, and GPU-fused approaches.
- Steps 8–10: Add sweep runner, comparisons, plots, animation, live demo, and report table.
- Steps 10–12: Run correctness tests, including fusion equivalence.
- Steps 13–21: Run approaches and full sweep, display results, plots and animation, run the demo, and package everything into `results.zip`.

2.2 Software and Hardware Setup

- All experiments were conducted on Google Colab using Python 3.13 and an NVIDIA Tesla T4 GPU.
- We used three libraries:
    - NumPy: To handle arrays on the CPU.
    - Numba-CUDA: To run our code on the GPU using `@cuda.jit`.
    - Matplotlib: To create graphs and animations of the results.

2.3 Hyperparameters Used in code

Sweep Settings

- **Number of games (N):** 11 values from 1 to 1,048,576 (2^0 to 2^{20}), each four times the previous one: 1, 4, 16, 64, 256, 1,024, 4,096, 16,384, 65,536, 262,144 and 1,048,576.
- **Steps per iteration (K):** 1, 10, 100 and 1,000.
- **Samples per run:** N × K, from a single sample up to roughly 1.05 billion.
- **Approaches**: CPU loop, GPU vectorized (K launches), and GPU fused (1 launch).
- **Random seed**: 42, used for all three approaches.

Both GPU approaches run every combination of N and K, giving 44 configurations per approach.

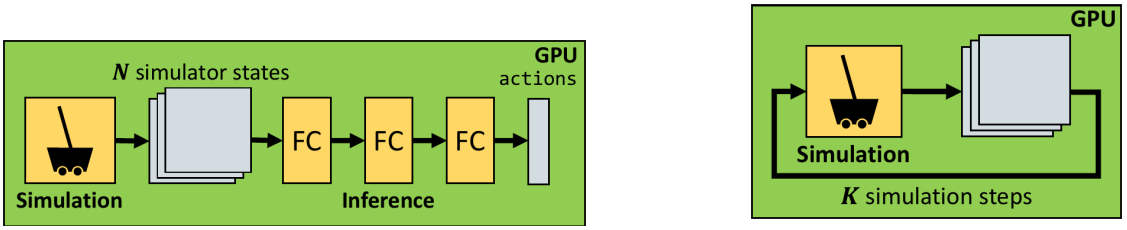
CPU related parameters:

- **Maximum N**: 262,144 games. The CPU runs all four K values for each N up to this limit, giving 40 configurations.
- **The CPU was not run at N = 1,048,576**, since pure Python is too slow at that scale (we estimate it would take about 1.9 hours).
- **Largest CPU run:** N = 262,144 and K = 1,000, which comes to about 262 million game-steps.

2.4 The three approaches at a glance

Table 1: The three approaches, for one data-collection iteration of N games and K steps

Approach	Runs on	Parallelism	Kernel Launches	State accesses per game
0. CPU baseline	1 CPU core, Python	None	0	– (CPU memory)
1. GPU vectorization	GPU, 1 thread/game	N games at once	K	2K (every step)
2. GPU kernel fusion	GPU, 1 thread/game	N games at once	1	2 (once each way)



(a) GPU vectorization: N simulator states processed in parallel *(b) Kernel fusion: K simulation steps inside one launch*

Figure 2: The two GPU optimizations. Reproduced from Gleeson et al. [1], Figures 3 and 7. In our notebook the inference block of (a) is replaced by random actions, as in the paper's simulation-only kernel.

3. Data Collection Approaches and Observations

3.1 Approach 1: Single-core CPU baseline

This is the baseline from the paper. A single CPU core runs a Python loop that advances N independent simulators, one after another: at every step, it visits each game in turn.

Algorithm 1: CPU baseline (src/cpu_baseline.py)

```
S0 = [InitialState(seed, i) for i in range(N)]  # Untimed
for r in range(R):
    S = copy(S0)
    start_timer()
    for t in range(K):
        for i in range(N):
            S[i], done = StepGame(S[i], i, t, seed)
    T[r] = stop_timer()
T = median(T)
throughput = N * K / T
```

Observations

- It processes around 1.5×10^5 samples/s, with a median of 6.8 µs per game-step. Best throughput was 1.69×10^5 samples/s at $N = 16,\ K = 10$.
- The largest CPU run ($N = 262,144,\ K = 1,000$) took about **30 minutes**. The $N = 2^{20}$ configuration was skipped because it was estimated to **take around 2 hours.**
- For $N = 1,\ K = 1$, the CPU finished in 0.031 ms, outperforming both GPU versions because it avoids kernel-launch overhead.
- CPU and GPU final states differed by at most 4.77×10^{-7}, likely due to small rounding differences in `sin` and `cos` calculations.

3.2 Approach 2: GPU vectorization

Here the N game states live in an N × 4 float32 array in the GPU's global memory, and each GPU thread looks after one game. Each step is a separate kernel launch. An iteration therefore issues K of them back to back, with just one synchronization at the end.

Algorithm 2: GPU vectorization (src/gpu_vectorized.py)

```
def step_kernel(S, t, seed):            # runs once per game, all in parallel
    i = thread_id()
    if i < N:
        state = S[i]                    # read from GPU memory
        state = step_game(state, i, t, seed)
        S[i] = state                    # write back to GPU memory


S = gpu_array(N, 4)
init_kernel(S, seed)
sync()

start_timer()
for t in range(K):
    launch(step_kernel, S, t, seed)     # K launches
sync()
stop_timer()
```

Observations

- For N up to 4,096 run time barely changes with N but grows in step with K, which means the vectorized version is held back by kernel launches than by the GPU itself.
- Each launch costs 29 to 60 µs, most of which is probably Numba dispatching the launch from Python.
- Some measurements look noisy: at K = 10 for example N = 256 took 0.60 ms while N = 1,024 took 0.34 ms.
- Every step. Writes the full state 32 bytes per game so large runs end up limited by memory bandwidth.
- At N = 262,144 and K = 1,000 this traffic adds up to 8.39 GB. The T4's peak bandwidth of 320 GB/s puts a bound of 26.2 ms on N and K traffic and our measured 32.7 ms (about 256 GB/s) sits close to N and K traffic floor.
- Throughput peaks at 8.01 × 10^9 samples/s around N = 262,144 and then dips slightly to 7.70 × 10^9 at N = 2^{20}.
- Memory capacity is never a concern: even a million games fit in 16 MB.

Table 2: State traffic and measured times at K = 1,000 (traffic = N × 32 bytes × K)

N (games)	State traffic, vectorized	Measured vectorized time	State traffic, fused	Measured fused time
65,536	2.10 GB	29.7 ms	2.1 MB	3.58 ms
262,144	8.39 GB	32.7 ms	8.4 MB	12.4 ms
1,048,576	33.55 GB	136.2 ms	33.6 MB	42.5 ms

3.3 Approach 3: GPU kernel fusion

The fused version is similar to the earlier one, where each thread handles one game. The only change is that a thread now runs all K steps without launching the kernel again and again. It reads the game state at the start, keeps updating it until all steps are done, and stores the final result once.

Algorithm 3: GPU kernel fusion (src/gpu_fused.py)

```
def fused_kernel(S, K, seed):           # runs once per game, all in parallel
    i = thread_id()
    if i < N:
        state = S[i]                    # read from GPU memory ONCE, into registers
        for t in range(K):              # all K steps stay in registers
            state = step_game(state, i, t, seed)
        S[i] = state                    # write back to GPU memory ONCE


S = gpu_array(N, 4)
init_kernel(S, seed)
sync()

start_timer()
launch(fused_kernel, S, K, seed)        # ONE launch for all K steps
sync()
stop_timer()
```

Observations

- At K = 1, a fused iteration takes just 0.037 to 0.047 ms for every N up to 16,384. That is essentially the cost of one launch plus one synchronization, which the vectorized version pays K times.
- At N = 2^{20} and K = 1,000 the kernel takes 42.5 ms. For context, the 33.6 MB of state it moves would take only about 0.1 ms at the T4's peak bandwidth, so almost all of this time is spent on arithmetic rather than on memory traffic.
- In other words, the fused kernel keeps the compute units busy rather than waiting on memory. This is an inference from the timings, not a profiler result.

4. Results

4.1 Experimental conditions and metrics

All numbers below come from one complete run of the notebook on a Colab Tesla T4 (`results/timing.csv`): 44 configurations for each GPU approach and 40 for the CPU. **Throughput** is the number of samples collected, N × K, divided by the median collection time. **Fusion speedup** is reported in two ways: at equal K (vectorized time divided by fused time), and in the paper's style (fused throughput at K = 1,000 divided by fused throughput at K = 1). **Speedup over the CPU** is the CPU time divided by the GPU time for the same N and K.

Table 3: Head-to-head comparison at the two largest configurations (median times)

Configuration	Approach	Launches	Time (ms)	Throughput (samples/s)	Speedup vs CPU	Speedup vs vectorized
N = 262,144	CPU (1 core)	0	1,778,429	1.47×10^5	1×	–
K = 1,000	GPU vectorized	1,000	32.72	8.01×10^9	54,347×	1.0×
(262 M samples)	**GPU fused**	**1**	**12.36**	**2.12×10^{10}**	**143,842×**	**2.6×**
N = 1,048,576	CPU (1 core)	–	not run (≈ 2.0 h est.)	–	–	–
K = 1,000	GPU vectorized	1,000	136.23	7.70×10^9	–	1.0×
(1.05 B samples)	**GPU fused**	**1**	**42.52**	**2.47×10^{10}**	–	**3.2×**

Table 4: Selected results at K = 1,000 for smaller N (times in ms; larger N are in Table 3)

N (games)	CPU	GPU vectorized	GPU fused	Fusion speedup (equal K)	Fused vs CPU
1	6.73	28.99	1.011	28.7×	7×

N (games)	CPU	GPU vectorized	GPU fused	Fusion speedup (equal K)	Fused vs CPU
64	436.9	30.15	1.190	25.3×	367×
4,096	27,851	30.14	0.783	38.5×	35,585×
16,384	112,164	30.27	1.136	26.7×	98,775×
65,536	446,820	29.71	3.582	8.3×	124,724×

Figure 3 plots throughput for every configuration. The CPU line stays flat at about 1.5×10^5 samples per second (except for the very smallest runs), while both GPU lines climb almost linearly with N until the GPU fills up, at around N = 2^{18} for the vectorized lines and at around N = 2^{14} to 2^{16} for the fused lines at K = 100 and 1,000. The gap between the fused and vectorized lines grows with K for N up to 4,096, as expected if the vectorized version pays its launch cost once per step.

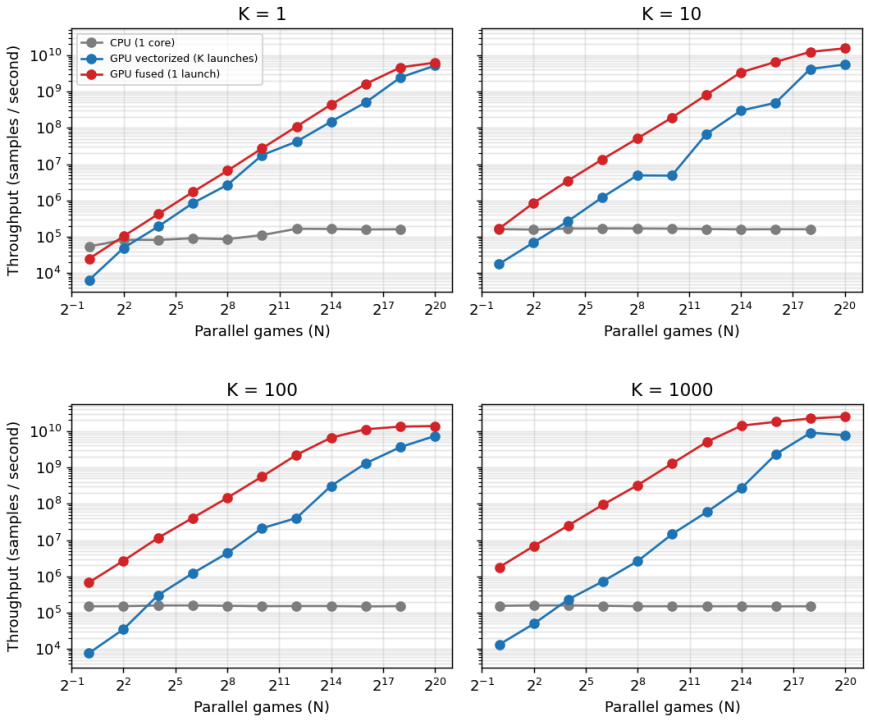
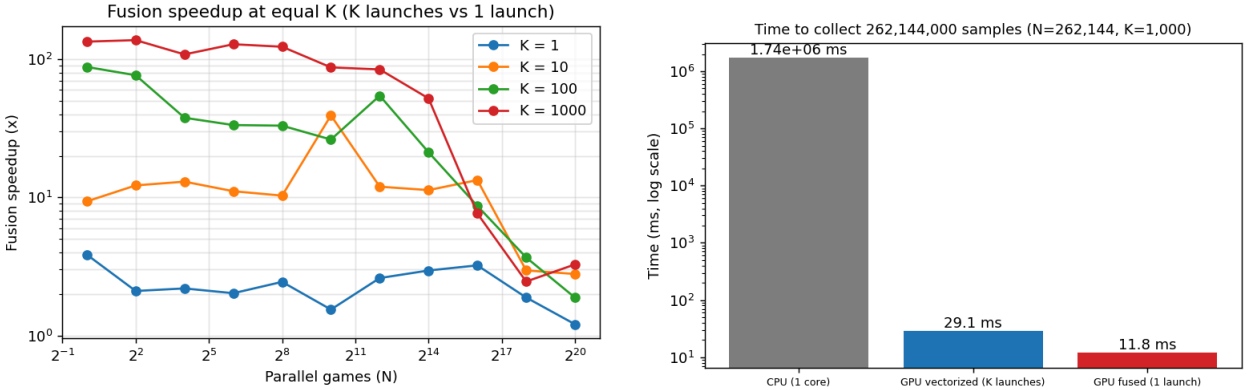


Figure 3: Data-collection throughput against the number of parallel games N, one panel per K (log–log axes). Grey: CPU; blue: GPU vectorized; red: GPU fused. Notebook output (`plot1_throughput_vs_N.png`), with its four panels arranged in a 2 × 2 grid for legibility.



(a) Fusion speedup at equal K versus N *(b) Time for 262,144,000 samples (N = 262,144, K = 1,000)*

Figure 4: Notebook outputs `plot3_fusion_speedup.png` and `plot2b_time_bars_with_cpu.png`. Both use logarithmic vertical axes.

- Fusion helps more as K grows and less as N grows.
- At N equals 4,096 and K equals 1,000 the fused version is 25 to 39 times faster than the vectorized version.
- After about N equals 65,536 launch stops being important: the speedup goes down to 2.6 to 3.2 times at the two largest sizes, which shows the benefit comes from less memory traffic instead of fewer launches.
- In all 44 configurations the equal-K speedup varies from 0.9, to 39.4 times.
- Figure 4(b) shows the three approaches at the largest size the CPU was run on (N = 262,144, K = 1,000):
    - CPU: 1,778,429 ms
    - Vectorization: 32.7 ms
    - Fusion: 12.4 ms

5. Analysis of the Best Approach

GPU kernel fusion is clearly the strongest approach. It was the fastest in 40 of the 44 configurations. It beat vectorization in 40 of 44; the four exceptions are all at K = 1 (N = 1, 256, 1,024 and 4,096), where both versions make one launch and differ by at most 0.006 ms. It beat the CPU in 39 of 40; the one exception is a single game for a single step. Its advantages come from how it uses the hardware.

1. **It removes the per-step launch overhead.** One launch replaces K. Each Numba launch costs roughly 29 to 60 µs of CPU work, so at K = 1,000 and N ≤ 4,096 the vectorized version spends between 29 and 31 ms on an iteration, most of it launching kernels, while the fused version finishes in 0.8 to 1.2 ms.
2. **It cuts global-memory traffic by a factor of K.** Each game's 16-byte state stays in registers, the fastest storage on the GPU, for all K steps. Global memory is touched twice per game instead of 2K times, which at $N = 2^{20}$ and K = 1,000 means 33.6 MB instead of 33.6 GB. A workload that was limited by memory bandwidth becomes limited by arithmetic instead.
3. **It scales the furthest.** Fused throughput was still rising at the largest size tested (2.47×10^{10} samples per second), while vectorized throughput leveled off at 8.01×10^{9} and then dropped slightly.
4. **It gives exactly the same results.** Fusion changes only the order of work, so with counter-based random numbers and independent games the results are bit-identical (confirmed in all 44 configurations). It can therefore be adopted without re-validating the simulator.
5. **It is cheap at small sizes and simple to write.** With a fixed cost of about 0.04 ms, the fused kernel beats the CPU from N = 1 for every K ≥ 10, whereas the vectorized version needs at least 16 games. The change from Approach 1 is also tiny: one loop moves inside the kernel, and the same device functions are reused.

6. Future Scope

- **Fuse the policy into the kernel.** This is the step the paper leaves open. A small network, such as a 4–64–2 MLP, fits in shared memory, so each thread could choose its own action and avoid the CPU–GPU round trip. Writing each step's observation, action and termination flag to memory would let the kernel feed an algorithm such as PPO [7]. The time to reach a target reward would then show the real end-to-end gain, which Amdahl's law suggests is smaller than the collection speedup.
- **Separate launch overhead from memory effects.** Launching the unfused baseline from C++ or a CUDA Graph would do this, and would test the claim that the fusion speedup does not depend on N. Nsight Compute could show occupancy and register use directly.

- **Try richer simulators.** Examples are the robotics environments in Brax [8], where contacts and diverging control flow make fusion harder, and network-defense simulators used to train automated cyber-defense agents.

7. Conclusion

The notebook reproduces the main findings of Gleeson et al. with three short, hand-coded implementations. GPU vectorization raised CartPole data-collection throughput from about 1.5×10^5 samples per second on one CPU core to 8.0×10^9, and kernel fusion raised it again to 2.5×10^{10}. The paper-style fusion speedup of 11.5 at N = 65,536 is very close to the paper's 11.3. The one claim we could not reproduce is that the fusion speedup does not depend on N, which we attribute to launching kernels from Python.

Kernel fusion is the best approach because it removes both of the costs that hold vectorization back, kernel launches and global-memory traffic, and it does so while returning exactly the same results. Since data collection is a large share of RL training time, this translates directly into shorter training runs, and in-kernel policy inference would be the natural next step toward an RL pipeline that runs entirely on the GPU.

References

[1] J. Gleeson, D. Snider, Y. Yang, M. Gabel, E. de Lara, and G. Pekhimenko, "Optimizing Data Collection in Deep Reinforcement Learning," in Proc. Third Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware (MLBench 2022), Santa Clara, CA, USA, 2022. arXiv:2207.07736.

[2] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," IEEE Trans. Systems, Man, and Cybernetics, vol. SMC-13, no. 5, pp. 834–846, 1983. doi:10.1109/TSMC.1983.6313077.

[3] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in Proc. Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15), ACM, 2015. doi:10.1145/2833157.2833162.

[4] NVIDIA Corporation, "NVIDIA T4 Tensor Core GPU Datasheet," 2019. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf

[5] M. Towers et al., "Gymnasium: A Standard Interface for Reinforcement Learning Environments," arXiv:2407.17032, 2024.

[6] C. Wellons, "Prospecting for Hash Functions," 31 July 2018. https://nullprogram.com/blog/2018/07/31/

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[8] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, "Brax: A Differentiable Physics Engine for Large Scale Rigid Body Simulation," in NeurIPS Datasets and Benchmarks Track, 2021. arXiv:2106.13281.